四层架构访谈准备

面向对象：

1. 需要和产品同事讨论 `~/Downloads/spec-四层架构.md`
2. 对项目还不熟，但希望能听懂大方向的人
3. 需要提前准备高优先级问题答案的人

整理日期：2026-04-14

主要依据：

1. `~/Downloads/spec-四层架构.md`
2. `AGENTS.md`
3. `docs/cad_iteration/INDEX.md`
4. `docs/cad_iteration/SYSTEM_RECORD.json`
5. `docs/cad_iteration/ITERATION_PROTOCOL.md`
6. `docs/cad_iteration/TOOL_SURFACE.md`
7. `src/sub_agent_runtime/agent_loop_v2.py`
8. `src/sub_agent_runtime/tool_runtime.py`
9. `src/sub_agent_runtime/feature_graph.py`
10. `src/sandbox/mcp_runner.py`
11. `src/sandbox_mcp_server/service.py`

一、先说结论

这份“四层架构”整理，整体上是好的，原因是：

1. 她已经抓住了这套系统最重要的定位：这是一个“建模执行子运行时”，不是直接面向最终用户的聊天产品。
2. 她把“上游调用视角”和“内部运行时视角”分开了，这一点非常重要。
3. 她已经意识到几个真正值得问的问题：上游边界、turn lane、session、repair packet、preflight lint。这几个确实是最容易让非开发同学看不懂，但又最影响产品判断的点。

如果要一句话评价她这份整理：

这份整理的方向是对的，已经不是“完全外行式理解”了；但里面有几处地方把“真实实现”说得有点太顺、太直线化了，所以访谈时最好把这些地方单独拿出来校准。

二、这份四层架构哪里整理得好

1. Layer 1 的定位是对的

她把 Layer 1 定义成“调用者集成层”，而不是“最终用户体验层”，这是对的。

大白话解释：

这个仓库不是让最终用户直接输入“帮我画一个盒子”然后出图的那层。
它更像是一个专业加工车间。前面已经有人把需求整理得差不多了，这个车间才开始真正干活。

所以：

1. 用户怎么说需求，不归它管。
2. 用户界面怎么展示，不归它管。
3. 它最关心的是：给我一个清楚点的 requirement，我能不能把 CAD 做出来。

2. Layer 2 抓到了 V2 loop 的核心节点

她已经把这些关键节点串出来了：

1. `IterativeSubAgentRunner`
2. `IterativeAgentLoopV2`
3. `V2ContextManager`
4. `ToolRuntime`
5. `McpSandboxRunner`
6. `DomainKernelState`
7. `Preflight Lint`

这说明她不是只看表面文件名，而是基本理解了：

1. prompt 不是直接拼出来就完事
2. tool call 不是直接打到底层
3. 语义状态和几何状态不是同一个东西

3. Layer 3 和 Layer 4 的拆法也基本合理

她已经意识到系统里至少有三类“状态”：

1. 运行态状态
2. 语义状态
3. 磁盘工件

这比“系统只有一个 state”要强很多。

因为这个项目里最容易混淆的，就是下面三种东西：

1. `RunState` 这种运行时记忆
2. `DomainKernelState` 这种语义图
3. `trace/` 、 `summary.json` 这种落盘工件

她能分出这些层，说明思路已经接近工程真实结构了。

三、这份四层架构哪里建议改

下面这些点，不是说她写错得很离谱，而是“如果不修正，访谈时很容易越聊越偏”。

1. Layer 1 里把输出写成了 `run_dir + summary + artifacts` ，这里要更准确一点

更准确的说法应该是：

1. `IterationRunResult` 直接返回的是 `run_dir + summary + request`
2. 真正的 artifacts 在 `run_dir` 对应的目录下面
3. 上游如果想拿渲染图、STEP、trace，不是从一个大 JSON 里直接拿，而是去对应目录读

大白话解释：

这个系统不是“接口直接把所有结果打包给你”。
它更像“我把加工完的东西都放到一个文件夹里，再告诉你这个文件夹在哪儿”。

2. Layer 1 里“轮询或等待完成”容易让人误会

当前代码里， `IterativeSubAgentRunner.run(...)` 是一个异步函数，但它不是自带“中途进度回调”的那种产品接口。

所以最好把这句话改成：

1. 上游可以异步等待整个 run 完成
2. 如果要做轮询或进度展示，需要上游自己包装

大白话解释：

它本身更像“一次跑完给结果”，不是“天然就会一边跑一边给你进度条”。

3. Layer 1 里 “Deliver 用户渲染图 + STEP” 也要补一句

当前仓库会产生：

1. STEP 文件
2. render 图
3. trace 和 summary

但“怎么变成给用户看的下载链接、URL、前端卡片”，是上游系统的事。

所以最好写成：

1. 本仓库产出本地工件
2. 上游负责把这些工件转成最终交付形式

4. Layer 2 的流程图看起来太像一条固定流水线

真实情况更像：

1. 每轮都先构造上下文
2. 然后让模型选工具
3. 但是运行时会根据当轮 policy 限制可选工具
4. 也就是说，不是每一轮都自由经过所有节点

尤其是 `turn lane` 不是固定写死的几条“人工流程”，而是运行时根据状态自动收窄工具面。

大白话解释：

不是说系统内部有个员工拿着流程卡：“第 1 步一定做 A，第 2 步一定做 B”。
更像是交通警察每个路口都根据当前情况决定：这轮只允许走哪几条路。

5. Layer 2 里 Turn Lanes 的表格还不够准确

她现在写的是：

1. code_first_build
2. code_repair_after_validation_blocker
3. local_finish
4. kernel_refresh
5. finish_or_continue

这个名称本身没问题，但真正需要补的一句是：

这些 lane 不是“开发者人工选择的工作流模板”，而是 `_determine_turn_tool_policy(...)` 每轮根据状态计算出来的策略结果。

这个区别非常重要。

因为产品同事容易以为：

1. lane 是一组业务流程配置

但工程真实情况更像：

1. lane 是运行时针对当前局势给模型下的“交通管制”

6. Layer 3 的数据流图有一个方向容易误导

她写的是：

```
RunState -> Prompt Bundle -> DomainKernelState -> Tool Records
```

更准确一点的理解应该是：

1. `RunState` 里本来就持有 `DomainKernelState`
2. Prompt bundle 会读 `RunState` 和 `DomainKernelState`
3. 工具结果出来后，再去同步 `DomainKernelState`
4. 不是 prompt bundle “生成” 了 domain kernel

大白话解释：

`DomainKernelState` 不是 prompt 临时拼出来的说明书。
它本身就是运行时一直维护的一个“语义账本”。

7. Layer 3 里 `RunState` 的示意结构不够准确

她写了：

1. `turns[]`
2. `lane`
3. `decision_summary`
4. `tool_calls[]`
5. `evidence_index`

这里最容易误导的点是：`lane` 不是 `TurnRecord` 里一个简简单单的固定字段。
更准确的说法是：

1. `TurnRecord` 记录该轮做了什么
2. `TurnToolPolicy` 记录该轮被允许做什么、偏向做什么

也就是说：

1. “已经发生的事情”
2. “这一轮被允许的行动空间”

是分开放的。

8. Layer 3 里工件文件名可以再对齐真实实现

例如她写了：

1. `prompts/round_XX_raw.json`
2. `prompts/round_XX_compacted.json`

但当前仓库里更常见、也更稳定的命名是：

1. `prompts/round_XX_request.json`
2. `prompts/round_XX_request_full.json`
3. `prompts/round_XX_user_prompt.txt`

这类小地方不一定影响理解，但访谈时如果拿去对照真实目录，最好还是统一。

9. Layer 3 里同步/异步延迟表，建议标成“经验估计”，不要写得像硬 SLA

她列了很多时间：

1. prompt bundle `<500ms`
2. LLM `<10s`
3. rendering `<120s`

这类数字最好写成：

1. 经验量级
2. 非协议承诺
3. 会受 provider、模型、渲染后端、负载影响

不然产品同事会误以为这是系统对外保证。

10. Layer 4 里要强调一点：preflight lint 不是独立服务

它不是一个单独部署的检查系统。
它更像 `ToolRuntime` 里对 `execute_build123d` 的写前拦截器。

大白话解释：

它不是“另一套系统来审代码”，而是“在真正把代码扔进沙盒前，先做一次便宜而确定的拦截”。

四、最高优先级问题详细解答

下面这部分，是最适合访谈时直接拿来讲的。

每个问题我都分成三层：

1. 一句话版本
2. 大白话版本
3. 稍微准确一点的工程版本

1. 上游 Main Agent 的职责边界到底是什么

一句话版本

上游 Main Agent 负责把需求讲清楚，这个仓库负责把已经讲清楚的需求真正做成 CAD，并在做的过程中自己修修补补直到收敛。

大白话版本

你可以把整个系统想成两拨人接力：

1. 第一拨人负责跟用户聊明白“你到底想要什么”
2. 第二拨人负责真正拿工具开工，把东西做出来

这个仓库属于第二拨人。

所以它不擅长，也不打算做这些事：

1. 跟用户来回确认模糊需求
2. 猜用户的偏好
3. 做前端交互
4. 把最终结果包装成产品级展示页

它更关心的是：

1. 这个 requirement 到底怎么建模
2. 第一次建模失败后怎么修
3. 怎么证明现在已经差不多做对了

稍微准确一点的工程版本

从代码和文档看，边界很清楚：

1. `IterationRequest.requirements` 被假设为“上游已归一化的结构化需求”
2. `docs/INTERFACE.md` 明确写了： `requirements` 应该已经 coherent
3. `AGENTS.md` 和 `docs/cad_iteration/*` 也明确把本仓库定位为 iterative CAD generation runtime

所以本仓库的输入不是“原始用户自然语言”，而是“已经过上游收敛后的 requirement payload”。

访谈时最容易被问到的追问

追问 1：如果上游 requirement 还是不清楚怎么办

大白话回答：

那这个仓库也许还能试着做，但它不是为了“继续澄清需求”设计的。
它更像一个加工车间，你给它一张模糊图纸，它可能会返工，但不会变成产品经理来帮你补需求。

追问 2：那它会不会自己偷偷补全需求

会有一些推断，但那是为了继续建模，不是为了承担上游澄清职责。
比如它会在 repair 时根据现有证据判断下一步怎么修，但不会把自己变成完整的需求理解系统。

2. turn lane 自动选择机制到底是什么

一句话版本

turn lane 不是人工手选流程，而是运行时每一轮根据当前局势自动给模型划定“这轮你只能走哪些路”。

大白话版本

很多人看到 `code_first_build` 、 `local_finish` 这些名字，会以为系统内部有几条固定流水线，人手工选一条走。

其实更像这样：

1. 每一轮开始时，运行时先看当前情况
2. 看你是不是首轮
3. 看上一轮是不是刚写成功
4. 看 validation 说是“确实有 blocker”还是“证据还不够”
5. 看前几轮是不是一直在失败
6. 看还剩几轮预算
7. 看 domain kernel 里有没有已经整理好的 repair patch / repair packet

然后它再决定：

1. 这轮只允许继续写代码
2. 还是只允许做 semantic refresh
3. 还是只允许局部 finishing
4. 还是可以直接 finish

所以 lane 更像“交通管制”，不是“项目经理写好的固定流程图”。

稍微准确一点的工程版本

这块核心在 `src/sub_agent_runtime/agent_loop_v2.py` 里的 `_determine_turn_tool_policy(...)` 。

它会返回一个 `TurnToolPolicy` ，里面至少包含：

1. `policy_id`
2. `mode`
3. `allowed_tool_names`
4. `blocked_tool_names`

5. `preferred_tool_names`
6. `preferred_probe_families`

也就是说，运行时不是只给模型一个建议，而是直接收窄它当轮能看到的工具面。

它主要参考哪些信号

最重要的信号包括：

1. 是否首轮
2. 最新成功写入是什么工具
3. 最新 validation 是否 fresh
4. validation 是“有 blocker”还是“证据不足”
5. blocker 是局部 finishing 尾巴，还是整体还没做对
6. 是否已经做过 semantic refresh
7. 是否已经连续多轮 `execute_build123d` 失败
8. 剩余 rounds 是否紧张
9. domain kernel 里是否已有 actionable patch / packet

最重要的几个典型 lane

`code_first_build`

首轮默认只允许 `execute_build123d` 。
意思是：先别搞局部动作链，先把一个能站住的整体实体做出来。

`local_finish`

当整体实体已经对了，只差局部 fillet / chamfer 这种尾巴时，才会偏向 `apply_cad_action` 。

`kernel_refresh`

当 validation 明确说“现在证据还不够”，或者连续 inspection-only turn 没有推进时，会强制先做语义刷新，而不是继续瞎修。

`code_repair`

当已经有明确 blocker，或者失败已经被归类成具体代码问题时，会优先继续 `execute_build123d` 修代码。

一个最容易听懂的类比

如果把系统比作医生看病：

1. lane 不是不同科室各自排队
2. lane 更像导诊台每一轮都在看：
   这人现在该去拍片、复查、开药，还是可以直接出院

访谈时最容易被问到的追问

追问 1：是不是模型自己决定 lane

不是。
模型决定“在允许的工具面里怎么选”，但 lane 是运行时先算出来，再用它收窄可选工具。

追问 2：为什么不全部交给模型自己判断

因为纯靠模型自己判断，容易出现：

1. 该修代码时一直去读 topology
2. 该 refresh 时继续盲目重写
3. 剩最后一轮还在做低价值读取

运行时 lane 的作用，就是把这些低价值游走压掉。

3. session 的生命周期管理到底是什么

一句话版本

session 是这次建模过程在沙盒里的“工作台状态”，默认一轮 run 对应一个 session，但如果调用方显式传同一个 `session_id` ，它也可以被继续读回。

大白话版本

你可以把 session 想成一个临时工作台。

每次建模不是凭空开始的。
系统需要记住：

1. 之前已经做了哪些 action

2. 当前模型长什么样
3. 现在的 step 是多少

这些东西就放在 session 里。

默认情况下：

1. 如果上游没给 `session_id`
2. 运行时就自动生成一个新的
3. 这次 run 基本上就用这一个 session 从头跑到尾

所以对大多数人来说，最简单的理解就是：

一场 run，有一个自己的工作台。

稍微准确一点的工程版本

session_id 从哪里来

在 `agent_loop_v2.py` 里：

1. 如果 `request.session_id` 有值，就用它
2. 否则自动生成 `iter-v2-<uuid>`

session 里存什么

在 `src/sandbox_mcp_server/service.py` 里， `SessionState` 主要存：

1. `history`
2. `current_step`
3. `created_at`

`SessionManager` 会：

1. 内存里保一份
2. 同时往磁盘 `/tmp/aicad_mcp_sessions` 落一份

为什么 query 工具能看到之前的状态

因为所有 query 工具都会吃同一个 `session_id` 。
只要写工具成功把状态 persist 进这个 session，后面的：

1. `query_geometry`
2. `query_topology`

3. `render_view`
4. `validate_requirement`

都会在同一个 session 上读。

session 会不会一直永久存在

严格说，不应把它理解成产品级长期存档。
它更像运行时的持久化工作状态：

1. 能在一段时间内复用
2. 甚至可以从磁盘恢复
3. 但它不是面向用户项目管理的正式持久化模型

还要补一个容易忽略的点

`apply_cad_action` 路径里，系统不是“只对当前结果做一个小补丁”那么简单。
它会根据 session history 重放整个 action history，重新构建模型代码，再执行。

大白话解释：

它不是在 CAD 软件里直接鼠标点一下某个边。
它更像“我拿着你之前的操作记录，重新按顺序做一遍，然后把新的这一步也加进去”。

访谈时最容易被问到的追问

追问 1：session 是不是等于用户会话

不是。
用户会话是产品概念。
这里的 session 更接近“沙盒里的建模状态上下文”。

追问 2：一个 session 能不能跨多次 run 复用

技术上可以，因为 `session_id` 是可传入的，底层也有磁盘恢复。
但当前 canonical 用法仍然更偏向“一次 run 一个主 session”，而不是把它当产品级项目状态。

追问 3：什么时候会 clear session

只有显式 reset 或某些需要强制清空状态的路径，才会做 `clear_session` 。
规范里也明确说了，不要把 clear_session 当常规操作。

4. repair packet 到底是什么

一句话版本

repair packet 是系统在“已经诊断出问题在哪、怎么修大致也知道”的情况下，生成的一张结构化修复卡片，比普通重写更窄、更可控。

大白话版本

先别把它想成什么复杂 AI 概念。
它其实就是：

1. 系统已经看出来某类 feature 出问题了
2. 也已经知道这个 feature 大概挂在哪个 host 上
3. 锚点和参数也有一些线索了

这时候如果还让模型完全自由发挥去重写，就很浪费。

所以系统会说：

“别整套重写了，我给你一张修理卡片。卡片上写着：

1. 修的是哪一类 feature
2. 修哪一个实例
3. 它的宿主是谁
4. 理想锚点是什么
5. 现在实际做成了什么样
6. 如果有稳定配方，连 recipe skeleton 都给你”

这张卡片，就是 repair packet。

稍微准确一点的工程版本

`FamilyRepairPacket` 目前会携带这些关键信息：

1. `family_id`
2. `feature_instance_id`
3. `repair_mode`
4. `host_frame`
5. `target_anchor_summary`
6. `realized_anchor_summary`

7. `recipe_id`
8. `recipe_summary`
9. `recipe_skeleton`

它来自 `DomainKernelState` ，并通过 `domain_kernel_digest` 的 `latest_repair_packet_*` 字段暴露给模型。

如果当前 packet 是运行时支持的类型， `ToolRuntime` 还可以直接开放 `execute_repair_packet` 。

repair packet 和普通 patch 有什么区别

可以这样理解：

patch

更像“修理方向说明书”

repair packet

更像“已经带着部分配方的修理工单”

也就是说：

1. patch 只告诉你往哪里修
2. packet 则尽量把“怎么修”也一起结构化了

为什么它重要

因为它代表系统正在从：

1. 完全自由重写

往

2. 家族化、结构化、低方差修复

这个方向走。

这对产品和工程都重要：

1. 更稳定
2. 更省轮次
3. 更容易解释

访谈时最容易被问到的追问

追问 1：repair packet 是不是所有问题都有

不是。
只有当系统觉得证据已经够具体，某些问题才值得收缩成 packet。

追问 2：它是不是一定会替代 execute_build123d

不是。
如果 packet 不支持、证据不够，或者修复范围还是太大，系统还是会退回 `execute_build123d` 。

追问 3：它现在是不是已经很成熟

它已经是明确方向，但仍在扩展中。
可以理解成：这不是概念图了，已经有真实实现，但支持面仍在逐步扩。

5. preflight lint 到底是什么

一句话版本

preflight lint 是在真正把 Build123d 代码扔进沙盒执行前，先做一次便宜、确定、可分类的拦截，避免把明明一眼就知道错的代码再浪费一轮去跑。

大白话版本

你可以把它想成“起飞前检查”。

系统本来要做的是：

1. 生成一段 Build123d 代码
2. 把代码送进沙盒
3. 看看能不能出结果

但很多错误，其实根本没必要跑进沙盒才知道，比如：

1. Python 语法错了
2. helper 名字写错了
3. 把 `CounterSinkHole(...)` 放错上下文了
4. 用了 Build123d 根本不支持的 keyword

如果这些错误还送进沙盒：

1. 浪费一整轮
2. 浪费 token
3. 还会让后面系统误以为是几何问题

所以 preflight lint 的本质就是：

先把“明明是代码写法错”的问题拦在门口。

稍微准确一点的工程版本

这块在 `src/sub_agent_runtime/tool_runtime.py` 里的 `_preflight_lint_execute_build123d(...)` 。

它会做几类检查：

1. Python 语法 / 缩进检查
2. AST 级别的上下文错误检查
3. 已知错误 helper / keyword / API surface 检查
4. 某些高频家族问题的 repair hint 生成

它不仅返回“失败”，还会返回：

1. `rule_id`
2. `message`
3. `repair_hint`

如果命中的家族比较稳定，还可能附带 compact repair recipe。

它拦的典型错误有哪些

例如：

1. `CountersinkHole(...)` 这种拼错 helper 名字
2. `CounterSinkHole(...)` 放进 `BuildSketch` 而不是 `BuildPart`
3. `Cylinder(..., axis=...)`
4. `Box(..., depth=...)`
5. `Semicircle(...)`
6. `subtract(...)`
7. `shell(...)`
8. `part.solid = part.solid - cutter`

9. 缺少最终 `result = ...` 或 `part = ...`

为什么产品同事也该关心这块

因为它会直接影响：

1. 系统是不是看起来“很笨”
2. 同样的问题会不会反复犯
3. 每一轮有没有被浪费

大白话说：

preflight lint 就是在防止系统犯那种“肉眼都知道不该犯”的低级错误。

访谈时最容易被问到的追问

追问 1：它和真正执行一遍代码有什么区别

区别在于成本和确定性。

1. 真执行要进沙盒，贵、慢、还可能带更多噪音
2. preflight lint 是便宜的前置拦截

追问 2：它会不会把本来能跑的代码误杀

理论上可能有误伤风险，所以它更适合拦那些高确定性的已知错误面。
从当前实现看，它主要瞄准的也正是这一类问题。

五、建议你在访谈时顺手补问的相关问题

下面这些不是她列的最高优先级，但通常会和那五个问题连在一起。

1. 为什么默认第一写是 `execute_build123d`，而不是 `apply_cad_action`

大白话答案

因为系统更想先快速拿到一个“整体站得住的实体”，而不是一开始就陷入局部动作链。

`apply_cad_action` 更像局部修边工具。
如果整体壳子还没搭起来，就先拿修边刀，很容易越修越乱。

你可以顺手问她

“你会不会把 `apply_cad_action` 理解成更轻量所以应该优先？如果会，这里正好可以聊清楚：它不是轻量第一写，而是局部 finishing 工具。”

2. 为什么系统强调“一轮最多一个写工具”

大白话答案

因为这样每次写完以后，后面的证据归因会更清楚。

如果一轮连写三次：

1. 到底是第几次写出了问题
2. 哪个 validation 是对应哪个版本
3. 哪个 render 看的是哪次结果

都会变得很乱。

所以“一轮一个写工具”本质上是在保护可调试性。

3. `DomainKernelState` 到底是不是第二个 CAD 引擎

大白话答案

不是。
它不是拿来真正建模的，它是拿来记“现在系统觉得还差什么、卡在哪里、建议怎么修”的语义账本。

真正建模还是靠：

1. `execute_build123d`
2. `apply_cad_action`
3. `execute_repair_packet`

4. 为什么 `validate_requirement` 不是每一轮都跑

大白话答案

因为它是“裁判”，不是“陪跑观众”。

如果每一轮都叫裁判上场：

1. 轮次会变贵
2. 信息会变重复
3. 有时候系统真正缺的是几何证据，不是再听一次总评

所以它更适合在：

1. 接近完成时
2. 或者已经明显卡住、需要判断下一步该不该停时

再出现。

5. 为什么系统这么强调 artifacts 可检查

大白话答案

因为这类 agent 系统不怕失败，怕的是失败以后不知道为什么。

这个仓库特意保留：

1. `prompts/`
2. `plans/`
3. `actions/`
4. `queries/`
5. `outputs/`
6. `trace/`

就是为了让任何一次失败都能复盘。

6. session 和 artifacts 的区别是什么

大白话答案

session 更像“工作台上的当前状态”，
artifacts 更像“这次加工过程中留下来的记录和产物”。

所以：

1. session 是为了继续干活
2. artifacts 是为了回头检查和解释

六、我建议你访谈时可以怎么开场

如果你自己对项目也不够熟，最稳的开场方式不是一上来就问细节，而是先用下面这套顺序：

开场顺序建议

1. 先确认边界

    - “我们先确认一下，这个仓库到底负责哪一段，不负责哪一段。”
2. 再确认主循环
    - “如果把一次 run 讲成人话，它每一轮到底在重复什么动作？”
3. 再确认五个高优问题
    - 上游边界
    - turn lane
    - session
    - repair packet
    - preflight lint
4. 最后再聊她图里那些更细的结构和术语

这样做的好处是：

1. 先把大图钉住
2. 再去讨论细节
3. 不容易因为一个术语卡住整场访谈

七、如果只给产品同事讲 3 分钟，可以怎么讲

你可以直接用下面这段大白话版本：

这个仓库不是负责跟用户聊天的，它更像 CAD 加工车间。
上游 Main Agent 先把需求整理清楚，然后把 requirement 交给这里。
这里会一轮一轮地做三件事：先看现在知道什么，再决定这轮该用什么工具，然后根据新证据判断继续修还是停。
为了不让系统乱跑，运行时每一轮都会自动限制它这轮能走哪些路，这就是 turn lane。
为了不把一眼就错的代码送进沙盒浪费轮次，写代码前还有 preflight lint。
如果系统已经很清楚某个 feature 怎么修，它不会每次都整套重写，而会尽量收缩成 repair packet 这种更窄、更可控的修复卡片。
整个过程中所有 prompt、动作、查询和结果都会落盘，所以出问题以后我们能追溯，不会只看到一句“失败了”。

八、给你的最终建议

如果你是以“我自己也不太懂”的状态去聊，我建议你把这场访谈的目标定成下面三件事，而不是试图一次性理解所有实现细节：

1. 先搞清楚系统边界
2. 再搞清楚一轮到底怎么推进
3. 最后搞清楚为什么它要有 session、repair packet 和 preflight lint 这些“看起来很工程”的东西

只要这三件事理解了，后面的术语基本都能挂得上去。

如果这三件事没理解，就算听懂了很多词，也很容易还是觉得这个系统“像黑箱”。